

Practice Set: Operator Overloading in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define **Operator Overloading** in C++. Explain how it provides syntactic elegance and natural mathematical intuition for user-defined types compared to named functions like `c1.add(c2)`.

Question 2: Explain the compile-time translation mechanism of operator overloading. How does the compiler map an expression like `c3 = c1 + c2`; to internal member function calls without incurring runtime performance overhead?

Question 3: Detail the roles of the **Left-hand Operand** and the **Right-hand Operand** when overloading a binary operator as a class member function. How are their respective data members accessed inside the operator function?

Question 4: Explain how a **Unary Operator** (e.g., unary minus `-c1`) is overloaded as a member function. Why does its member function declaration accept zero parameters?

Question 5: Describe the ambiguity problem encountered when overloading the increment operator (`++`). How does C++ resolve the signature conflict between **Pre-Increment** (`++i`) and **Post-Increment** (`i++`)?

Question 6: Detail the difference in execution behavior between pre-increment (`operator++()`) and post-increment (`operator++(int)`) in terms of internal temporary object creation and returned values.

Question 7: List the six C++ operators that **CANNOT** be overloaded under any circumstance.

Question 8: Explain the immutable constraints of operator overloading regarding:

- Operator Precedence and Associativity

- Operand Count

Inventing New Operators

Primitive Data Type Redefinition

Question 9: What is meant by **Canonical Operator Semantics**, and why should developers avoid overloading operators like + to perform non-intuitive tasks like subtraction or file deletion?

Question 10: Write a complete C++ program defining a Complex class that overloads:

The binary + operator to add two complex numbers.

The unary - operator to negate both real and imaginary components. Demonstrate their usage in main() with clean mathematical syntax.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): What keyword is required to declare an operator function in C++?

Question 12 (Viva): Does operator overloading introduce any runtime lookup or performance penalty compared to standard member function calls?

Question 13 (Viva): What internal function call does the compiler generate when it parses the post-increment expression `c1++`?

Question 14 (Viva): Why does the post-increment operator function include a dummy int parameter in its signature?

Question 15 (Viva): Can you overload the Scope Resolution Operator (`::`) or the Member Access Dot Operator (`.`)?

Question 16 (Viva): Can you create a custom operator like `**` for exponentiation or `@` for matrix operations in C++?

Question 17 (Viva): Can you overload an operator such that both operands are primitive data types (e.g., changing how `int + int` works)?

Question 18 (Viva): When a binary operator is overloaded as a member function, how many arguments appear explicitly in its parameter list?

Question 19 (Viva): Which implicit pointer binds the left-hand operand inside a member operator function?

Question 20 (Viva): Can you change the precedence or associativity of an operator through operator overloading?

Answers & Explanations

Answer: 01

Definition: Operator Overloading is a compile-time polymorphism feature in C++ that allows standard operator symbols (such as +, -, ++) to be redefined and used as function names for user-defined classes and structs.

Syntactic Elegance: Built-in operators work natively only on primitive types. Without operator overloading, combining user-defined objects requires verbose named member functions like `c3 = c1.add(c2);`. Overloading allows developers to write intuitive mathematical expressions like `c3 = c1 + c2;`, matching the natural syntax of primitive types.

Answer: 02

Translation Mechanism: Operator overloading is syntactic sugar resolved entirely at compile time.

Mapping: When the compiler parses `c3 = c1 + c2;`, it translates the expression directly into the member function invocation `c3 = c1.operator+(c2);`. Because this translation occurs during compilation, the machine code executed is identical to a standard member function call, incurring zero runtime performance overhead.

Answer: 03

Left-hand Operand (c1): Acts as the **Caller Object** that invokes the member function via its implicit `this` pointer. Its data members (a, b) are accessed directly without dot notation.

Right-hand Operand (c2): Is passed explicitly as the **Argument Object** into the function's parameter list (Complex c). Its members must be accessed via the parameter identifier using the dot operator (c.a, c.b).

Answer: 04

Mechanism: A unary operator operates on a single operand (e.g., -c1). When overloaded as a member function, the operand itself is the caller object invoking the method (c1.operator-()).

Zero Parameters: Because the single operand is already bound to the function through the implicit this pointer, no additional arguments are needed in the parameter list.

Answer: 05

Ambiguity Challenge: Both pre-increment (++i) and post-increment (i++) use the exact same operator symbol (++), making it impossible for the compiler to differentiate them by symbol alone.

Resolution Convention:

Pre-Increment: Declared with zero parameters: operator++().

Post-Increment: Declared with an unused dummy int parameter: operator++(int). When invoked as i++, the compiler automatically passes a dummy value of 0 (i.operator++(0)) to select the postfix routine.

Answer: 06

Pre-Increment (operator++()): Increments the internal data member directly and returns the updated object, requiring no temporary copies.

Post-Increment (operator++(int)): Preserves the original state by saving it in a temporary local object, increments the internal data member, and returns the old temporary object.

Answer: 07

The six operators that **cannot** be overloaded in C++ are:

. (Member Access Dot Operator)

.* (Pointer-to-Member Operator)

:: (Scope Resolution Operator)

?: (Ternary Conditional Operator)

sizeof (Compile-time Size Operator)

typeid (Runtime Type Identification Operator)

Answer: 08

Precedence and Associativity: Remain strictly fixed as defined by language grammar; you cannot make + take precedence over * or alter associativity.

Operand Count: Is immutable; a binary operator must take two operands, and a unary operator must take one.

No New Operators: Developers can only overload existing C++ operators and cannot invent new symbols (e.g., **, @, or \$).

User-Defined Requirement: At least one operand must be a user-defined class, struct, or enum; altering operators strictly for built-in primitive types (e.g., int + int) is forbidden.

Answer: 09

Meaning: Canonical operator semantics means maintaining the standard, intuitive mathematical behaviors associated with an operator symbol.

Software Integrity: Overloading + to perform subtraction or execute side effects like file deletion violates caller expectations, obscures system architecture, and introduces subtle, hard-to-diagnose bugs.

Answer: 10

```
#include <iostream>
using namespace std;
class Complex {private:
    int a, b;public:
    void setData(int x, int y)
    {
        a = x;
        b = y;
    }

    void showData()
    {
        cout << "a = " << a << " | b = " << b << endl;
    }

    // 1. Overloading Binary '+' Operator
    Complex operator+(Complex c)
    {
        Complex temp;
        temp.a = a + c.a; // 'a' is caller (c1), 'c.a' is argument (c2)
        temp.b = b + c.b;
        return temp;
    }
}
```

```

// 2. Overloading Unary '-' Operator (Negation)
Complex operator-()
{
    Complex temp;
    temp.a = -a;
    temp.b = -b;
    return temp;
}
};
int main()
{
    Complex c1, c2, c3, c4;
    c1.setData(3, 4);
    c2.setData(5, 6);

    // Binary addition: c1.operator+(c2)
    c3 = c1 + c2;
    cout << "Result of c1 + c2:" << endl;
    c3.showData(); // Outputs: a = 8 | b = 10

    // Unary negation: c1.operator-()
    c4 = -c1;
    cout << "Result of -c1:" << endl;
    c4.showData(); // Outputs: a = -3 | b = -4

    return 0;
}

```

Answer: 11

The **operator** keyword.

Answer: 12

No; it is resolved entirely at compile time via static binding, generating machine code equivalent to a standard member function call.

Answer: 13

c1.operator++(0);

Answer: 14

To provide a unique function signature that allows the compiler to distinguish postfix increment from prefix increment.

Answer: 15

No; both `.` and `::` are strictly non-overloadable in C++.

Answer: 16

No; developers can only overload existing C++ operators and cannot define new operator tokens.

Answer: 17

No; at least one operand must belong to a user-defined type (class, struct, or enum).

Answer: 18

Exactly one explicit argument, because the left-hand operand is implicitly passed as the caller object via this.

Answer: 19

The implicit **this pointer**.

Answer: 20

No; operator precedence and associativity are fixed by the C++ language standard and cannot be altered